

AI Intern Assignment - PDF Intelligence & Collaboration System

About This Assignment

This take-home assignment gives you a realistic feel for the kind of work we do at SpotDraft. We work extensively with PDFs, large language models, and real-time collaboration tools — this project touches all three.

Complete the features listed below, deploy your application, and share a working demo with us. We value depth over breadth: a smaller set of features that work cleanly is far better than all features barely working.

Timeline: You have **4 days** from the date of receiving this assignment to submit.

What to Submit

- A **publicly deployed, working application**: share the URL
- A **short video walkthrough** (3–5 minutes, Loom or similar) demonstrating all features, including the AI capabilities
- A **GitHub repository** (public, or private with access shared) with a README covering setup, environment variables, and how to run locally

Overview

Build a **PDF Intelligence & Collaboration System** — a web application where users can upload PDFs, get AI-generated summaries, ask questions about the content via an LLM-powered chat, share files with others, and collaborate through comments.

Must-Have Features

1. User Signup and Authentication

- Users can create an account with name, email address, and password
- Authentication must be implemented to ensure secure access to the application
- User passwords must be securely hashed and stored

2. File Upload

- Authenticated users can upload PDF files to the system
- The application must validate that uploaded files are in PDF format
- Uploaded PDFs are accessible only to the uploader and explicitly invited users

3. Dashboard

- Users can see all PDFs they have uploaded
- Users can search for PDFs by filename
- Each PDF entry on the dashboard displays the filename, upload date, and its AI-generated summary (see Feature 6)
- Clicking a PDF opens the full PDF viewer with the comment and chat panel

4. File Sharing

- Users can share a PDF by generating a unique shareable link
- Invited users do not need an authenticated account to access a shared PDF via its link

5. Invited User File Access and Commenting

- Invited users can access shared PDFs via their unique link
- A user-friendly interface should allow viewing the full PDF
- Both invited users and the authenticated owner can add comments in a sidebar panel

6. AI-Powered PDF Summary

- When a PDF is uploaded, the system must automatically:
 1. Extract the text content from the PDF
 2. Send it to an LLM API (your choice: OpenAI, Anthropic Claude, or Google Gemini) with a well-constructed prompt
 3. Store and display a 3–5 sentence summary
- The summary must appear on the dashboard card and at the top of the PDF view page
- Pay attention to your prompt design — the summary should be concise, accurate, and useful, not a generic restatement

7. AI-Powered PDF Chat

- In the PDF viewer, provide a chat interface where users (both authenticated and invited) can ask natural language questions about the PDF
- The system must:
 1. Extract and process text from the PDF
 2. Pass the user's question along with relevant PDF context to an LLM API
 3. Return a grounded, accurate answer based on the document content
- The chat must be conversational, maintain at least the last 3–5 turns of context so follow-up questions work naturally.
- Handle cases where the PDF is long and its full text cannot fit in a single context window — you may use chunking, summarization, or any other strategy; document your approach in the README.
- The LLM API used for chat can be the same as or different from the one used for summaries.

8. Security and Data Privacy

- Access controls must ensure only authorized users can access PDFs and their comments
- User passwords must be securely hashed — do not store plaintext passwords
- LLM API keys must never be exposed in client-side code and must not be committed to the repository — use environment variables

9. UI and Design

- The application must have an intuitive, user-friendly interface
- Responsive design should support various screen sizes
- The PDF viewer, comment sidebar, and AI chat panel should be clearly laid out and easy to use

Good-to-Have Features

1. Authentication Enhancements

- Password reset and account recovery flow

2. Email Notification on Share

- Send an email to the invitee when a PDF is shared with them

3. Comment Enhancements

- Users can reply to existing comments (threaded comments)
- Basic text formatting in comments: bold, italic, bullet points

4. Semantic PDF Search

- Upgrade the dashboard search to use embedding-based semantic search — find PDFs by what they are about, not just by filename
- For example, searching "employment contract" should surface a PDF titled "Agreement_v3.pdf" if its content is about employment terms

5. Streaming AI Responses

- Stream LLM responses token-by-token in the chat interface for a real-time feel, rather than waiting for the full response before displaying it

Evaluation Criteria

We will evaluate your submission on the following dimensions:

- **Core engineering:** Auth works, file handling is clean, sharing and commenting function end-to-end

- **AI implementation:** Does the summary make sense? Does the chat answer questions accurately and stay grounded in the PDF? How well-constructed are your prompts? How do you handle long documents?
- **Code quality:** Clean, readable, well-structured code; no API keys in the repo; sensible separation of concerns
- **Deployment:** The application is publicly accessible and all features work in the deployed environment
- **Communication:** The README is clear and covers setup end-to-end; the video walkthrough demonstrates all features including AI capabilities

Guidelines and Notes

- You may use **any LLM API** — OpenAI, Anthropic Claude, and Google Gemini all have free tiers or trial credits sufficient for this assignment
- You may use **any tech stack** — Node.js, Python/FastAPI, Next.js, Django, etc. Pick what you are most productive with
- **Do not expose API keys** in frontend code or commit them to your repository. Use environment variables and add a .env.example file so reviewers know what to configure
- In your README, briefly describe your approach to the AI features — what LLM you used, how you structured your prompts, and how you handled long PDFs
- If you run into a blocker or need to make a scope trade-off, note it clearly in the README — we appreciate transparency

Deliverables Checklist

- ☐ Deployed application: public URL
- ☐ GitHub repository: public or with access shared
- ☐ Video walkthrough: 3–5 minutes covering all must-have features, including AI summary and chat (Loom or similar)

Good luck! If you have any questions about the requirements, feel free to reach out.